

Universidad de Guanajuato

Materia: Bases de datos no relacionales

Carrera: Licenciatura en Ingeniería de Datos e Inteligencia Artificial

Práctica 1:Montar API rest en DOCKER

Alumno: José Adrián Rodríguez González

NUA: 148661

Profesor: Dr. Yahir

18 de septiembre de 2025

Índice general

1. Introducción **2**

2. Desarrollo **3**
2.1. Creación de contenedores 3
2.2. Código 5
2.2.1. Inicialización del proyecto 5
2.2.2. Archivo de configuraciones 5
2.2.3. Archivo del modelo 6
2.2.4. Servicio 6
2.2.5. Controlador 7
2.2.6. Sección principal 9

3. Resultados **10**
3.1. El hola 10
3.1.1. 10

4. Conclusiones **12**

Capítulo 1

Introducción

En esta práctica se desarrolló un sistema de CRUD para una base de datos en una universidad, en la cual registrará estudiantes. Se usó Docker para orquestar los contenedores donde vivirá el servidor y la base de datos. Se utilizó una base de datos del tipo no relacional basada en documentos, mongodb. Y las tecnologías utilizadas para el desarrollo del servidor fueron javascript con node y el framework de express para hacer los endpoints.

Capítulo 2

Desarrollo

2.1. Creación de contenedores

Para esto se desarrollaron dos elementos, la parte de la base de datos y la parte del servidor. La base de datos como se mencionó en la introducción, fue mongodb. Esta base de datos va a estar dentro de un contenedor de docker. Sin embargo, tanto este contenedor como el de la API vivirán en contenedores temporales, por lo que se debe de hacer un volumen que permita la persistencia de los datos, sobre todo en la base de datos. Así que inicialmente podemos crear nuestro volumen por medio de los siguientes comandos

```
docker volume create mongo_db
docker network create practica_1
docker run --rm -d --name base_universidad --network practia_1 -p \
27017:27017 -v mongo_db:/data/db \
-e MONGO_INITDB_ROOT_USERNAME=root -e MONGO_INITDB_ROOT_PASSWORD=root \
-e MONGO_INITDB_DATABASE=universidad mongo:6.0
```

El primer comando sirve para crear un volumen con ese nombre, mongo_db, después se crea una red, por medio del segundo comando y con el nombre *practia_1.* Los volúmenes permiten que si el contenedor es temporal, la información almacenada se queda en un sector de memoria, y si se remueve el contenedor, ninguna información se pierde. Esto es práctico debido que evitamos tener almacenado contenedores inactivos. Por la otra parte, el segundo comando crea un switch virtual, esto implica que le asigna direcciones ip a cada uno de los elementos contenidos dentro de ese switch o que tengan esa conexión, a su vez de que tiene su propio enrutador. Solamente que ese switch se hace en base el localhost de nuestra máquina, por lo que si se desea por otro dispositivo externo, se debe primeramente,se deben de exponer los puertos de la comunicación principal, de esta manera si nos queremos conectar a esa dirección, podemos realizar mediante curl y colocar la ip que posee el dispositivo que monta los contenedores. Esta IP está asignada a un conexión de red, por lo que el dispositivo que desee conectarse al otro dispositivo, debe estar en la misma red,(si se desea realizar por ese método)

Tal que ahora tenemos el volumen y nuestro switch virtual, ahora se puede crear una base de datos. El tercer comando monta un contenedor temporal mediante rm, y el proceso de montado es d, por lo tanto, los logs se ven en segundo plano, con el

nombre de base_universidad, se le indica el nombre del switch virtual que previamente creamos, se desvelan los puertos para la comunicación 27017, y se coloca el nombre del volumen creado al cual, vivirá en el contenedor en la ruta descrita después de los ":". Se crean las variables de entorno que es el nombre de usuario, contraseña y el nombre de una base de datos inicial, finalmente la imagen que requerimos que es mongo en versión 6.0.

Para realizar el segundo se va a crear una imagen de dockerfile.

```
FROM node:24-alpine
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD [ "node","index.js" ]
```

Esta imagen primeramente necesitara la versión 24 de node, después configura como la raíz de trabajo */usr/src/app*. Se copian y pegan los archivos package*.json,(el asterisco es un comodín, por lo que no importa que es lo que siga o no siga en el texto, solo con ese formato de inicio es suficiente para identificar cuales son los archivos de nuestro interés). y los pega en la raíz de nuestro directorio de trabajo. Se ejecuta una instalación de aquellas dependencias que son de producción, esto quita aquellas que son de desarrollo. Después copia todo el proyecto a la raíz del directorio. Se expone el puerto 3000 y el comando por defecto que se ejecuta se determinar por CMD. Este comando es más que la inicialización del proyecto por medio de node y el archivo de index. Finalmente se monta la imagen con el siguiente comando

```
docker build -t mi-api .
```

Se construye y se nombra por medio *-t mi-api*, y busca la dirección en donde está tanto el docker file como los archivos que copiará al indicarle que es en esa raíz. Con esta imagen se puede realizar y levantar el contenedor con el siguiente comando

```
docker run --rm -d --name api_universidad --network practia_1 \
-p 3000:3000 -v $(pwd):/usr/src/app -v /usr/src/app/node_modules \
--env-file .env mi-api:latest
```

Con esté comando podemos crear y levantar el contenedor donde vendrá la API. Mediante rm se hace temporal, y de igual manera el log se hace en segundo plano por d. Se le nombra *api_ universidad* y se sube al switch que también se conectó nuestra base de datos. Después se levantan los puertos y (esta parte es opcional, ya que se usa cuando todavía está en desarrollo, si se pone en producción, solamente se debe de eliminar está sección). Al colocarle -v creamos un volumen que estará en nuestro directorio del proyecto, además de las dependencias. Esto con el fin de que si se realiza un cambio, se pueda ver reflejado sin tener que para el contenedor. En producción por lo general se omite. Se envían las variables de entorno, puede ser manualmente o enviarlo por un archivo .env. Finalmente se pide la imagen a usar, ya previamente construida. Una vez que ya están los dos, y el código fue desarrollado de manera correcta, se puede proceder a realizar pruebas. Pero antes de llegar al código se debe de explicar que hace.

2.2. Código

Como se mencionó en la introducción, se realizó un código en javascript con la arquitectura de modelo controlador. Durante el desarrollo de los contenedores, se mencionó que es necesario unas dependencias, y es que en un principio, para realizar el proyecto, se debió de haber instalado node y npm para poder realizar pruebas iniciales.

2.2.1. Inicialización del proyecto

Para ello con el comando *npm init*, se inicia un proyecto y con *npm init -y*, se pueden omitir los pasos de inicialización. Las dependencias necesarias son *express* y *mongoose*. El primero es un framework que permite el desarrollo de servidores y apis de manera más sencilla que realizarlo manualmente con node, y mongoose es un ORM que permite interactuar a javascript con mongo de una forma más simple. Los ORM por lo general ya tienen su abstracción de objeto para evitar que se deba de escribir código del lenguaje de la base de datos. Nuestro archivo principal será *index.js*, que contiene todas las llamadas para la base y la definición de endpoints. La raíz del proyecto está construido tal que

```
config/
controller/
Dockerfile
index.js
.env
model/
node_modules/
package-lock.json
package.json
reamde.md
student.service.js
```

Aquellos nombre que terminan con /, son rutas. El primero a observar es el Dockerfile, que es la imagen que montaremos al contenedor. El segundo es la raíz del proyecto, el tercero el archivo que contienen las variables de entorno. Los que terminan en json en esta caso, indican cuales dependencias se están utilizando, un readme con un pequeño resumen del proyecto y student.servce.js es un archivo que contiene los elementos o funciones que se usarán para la interacción de la base de datos, más adelante se explica en detalle este archivo.

2.2.2. Archivo de configuraciones

El primer archivo a observar para la interacción de la base de datos es la conexión. Aquí se importa el paquete de mongoose y se crean dos funciones, una que permite la conexión y la segunda la des-conexión de manera asíncrona. En la conexión es necesario pasarle las variables de entorno y crear nuestra url que permitirá la conexión a mongo. Por medio de un try catch, capturamos si hubieron errores en la conexión, ya que se intenta conectar y si fue correcto, imprime a que usuario se conectó de la base de datos. Si no, solo que no se pudo conectar. Finalmente una función que permite la desconexión. El uso de funciones asíncronas es debido a que muchos de los procesos que se

realizarán si no son exitosos, no queremos que nuestro programa concluya, quizás que brinde un mensaje de error, más no la detención del programa. Para ello, se utilizan las funciones asíncronas, o también por si una de las funciones tarda más, que la aplicación completa no quede congelada. Además de que las implementaciones desarrollados por este ORM son asíncronas, así que forzosamente las funciones que sean asíncronas deben de ser ejecutadas y contenidas en funciones que sean asíncronas. El ejemplo del código es el siguiente

Listing 2.1: Conexión a MongoDB con Mongoose

```
import mongoose from "mongoose";

export const connect = async () => {
    const {MONGO_USER, MONGO_PASS, MONGO_HOST, MONGO_PORT,
       MONGO_DB} = process.env;
    const url = `mongodb://${MONGO_USER}:${encodeURIComponent(
       MONGO_PASS)}@${MONGO_HOST}:${MONGO_PORT}/${MONGO_DB}?
       authSource=admin`;
    console.log(url)
    try {
        await mongoose.connect(url);
        console.log(`Conectado: ${MONGO_USER}`);
    } catch (error) {
        console.log("Error, no se pudo conectar");
    }
}

export const disconnect = async () => {
    await mongoose.disconnect();
    console.log("Se desconecto")
}
```

Este archivo se encuentra en la carpeta config con el nombre de db.js

2.2.3. Archivo del modelo

Después, pasamos a la sección de modelo, que define un schema, que es lo más parecido a las tablas en bases de datos relacionales. Contendrá un nombre que es un string, un variable requerida y que se recorten espacios en blanco. Una fecah de nacimiento del tipo Date siendo una variable requerida, y un correo electrónico único, en minúsculas y requerido. Finalmente una marca de tiempo de cuando se creo ese dato. Este schema se le denominará student. Este objeto tendrá métodos asociados, sin embargo solo utilizaremos aquellos que sean para el desarrollo del CRUD. Por lo tanto, este modelo va a ser exportado y utilizado en el archivo *student.service.js*.

2.2.4. Servicio

Dentro de el archivo, se definen 4 funciones, la esencia de un CRUD, crear, leer, actualizar y eliminar. Todas las funciones son asíncronas. Para la función de crear requiere una estructura del tipo estudiante, definida previamente en el modelo. Así que cuando se le pase por medio de la api, debe contener las propiedades que fueron

declaradas en el modelo. EL objeto Student, tiene asociado un método llamado create, y es el responsable que crea una nueva instancia o dato dentro de nuestro esquema. Así que create requiere el una variable o estructura del tipo Student. Algo muy similar sucede con get, que permite obtener todos los datos dentro de una colección. Este objeto ya tiene una función denominada find. Para el caso de update solo requerimos pasarle un filter, que es el valor que vamos a estar buscando para actualizar, y el student es el valor por el cual vamos a reemplazar. Para esto, el objeto también tiene un método qu permite actualizar. A manera de escalar, se uso únicamente al que actualiza 1 miembro que cumpla la condición, sin embargo el objeto posee también el método para actualizar a más de 1. Finalmente la función deletes, que recibe solo el estudiante o mejor dicho, un valor al cuál, sea partidario para ser eliminado, igual que update, solamente se eliminará un elemento.

Listing 2.2: Desarrollo inicial del CRUD

```
1   import { Student } from "./model/student.js";
2   export const create=async(student)=>{ //Create
3       return await Student.create(student);
4   }
5   export const get=async()=>{
6       return await Student.find();
7   }
8   export const update=async(filter,student)=>{
9       return await Student.updateOne(filter,student);
10  }
11  export const deletes=async(student)=>{
12      return await Student.deleteOne(student)
13  }
```

2.2.5. Controlador

Después se tiene una carpeta llamada controller en la que llamada las funciones que exporta services, y lo que realiza es verificar y checar que lo recibido por la url sea correcta, además de manejar los errores. Se podría usar directamente el services, pero es adecuado ponerlo en un wrapper que pueda identificar potenciales errores. Para ello, cada una de las funciones descritas tienen su propio manejador en una función propia, y cada una posee o requiere dos parámetros, un request y un response. El request es lo que irá recibiendo por medio de las peticiones que se realicen, mientras que el response, son las potenciales respuestas. Todo está manejado por try catch. En el caso de get, tiene su función *getStudents*, que llama a la función *get* y responde con un código 200 de que si dio un resultado, si no se pudo, se responde con 500 y da un pequeño mensaje de error. Para la creación de estudiantes es similar, solamente que si es necesario parámetros al acceder, entonces se busca que tena el nombre, la fecha de nacimiento y el correo, Si no es correcto ninguno se responde con 400. Igual, algunos valores como se mencionó son necesarios, así que o da error el mismo wrapper o al crearlo. Por lo tanto, se responde con un 201, por que se dio un nuevo estado y se envía el estudiante que fue creado. De nueva cuenta el error lo responde con 500 y un mensaje. Para actualizar un estudiante, se hace un proceso similar solamente que busca la existencia de filtro y estudiante. Ahora, si en base a lo que colocó, nada se actualizó,

entonces,eso implica que no existe alguna coincidencia, por lo tanto, se muestra una respuesta 404, de que no existe tal estudiante. SI la actualización fue exitosa, solamente es un código 200. Y al eliminar pasa algo parecido, se busca si existe el estudiante en el json enviado si no está, entonces error 400. Si al eliminar tampoco eliminó algo, error 404 porque entonces no existe tal estudiante. Si fue exitoso, devuelve un error 204.

Listing 2.3: Desarrollo inicial del CRUD

```
    import { get,create,update,deletes} from "../student.service
       .js";

export const getStudents =async(req,res)=>{
    try{

        const student =await get();
        res.status(200).json(student);
    }catch(error){
        res.status(500).json({
            message:"error al obtener estudiantes",
            error:error.message
        });
    }
}
export const createStudents =async(req,res)=>{
    try{
        const {name, birthDate, email}=req.body;
        if(!name ||!birthDate || ! email){
            return res.status(400).json({message:"Faltan campos"
               })
        }
        const student= await create({name,birthDate,email})
        res.status(201).json(student);
    }catch(error){
        res.status(500).json({
            message:"Error al crear al estudiante",
            error:error.message
        })
    }
}
export const updateStudent=async(req,res)=>{
    try{
        const {filter,student}=req.body;
        if(!filter || ! student){
            return res.status(400).json({message:"Faltan
               parametros"});
        }
        const result=await update(filter,student)
        if(result.modifiedCount===0){
            return res.status(404).json({message:"No se encontro
                ese estudiante"});
        }
```

```
40          res.status(200).json({message:"Estudiante actualizado",
               result})
41      }catch(Error){
42          res.status(500).json({ message: "Error al actualizar",
               error: error.message });
43      }
44  }
45  export const deleteStudent=async(req,res)=>{
46      try {
47          const {student}=req.body;
48          if(! student){
49              return res.status(400).json({message:"Faltan
                   parametros"});
50          }
51          const result=await deletes(student);
52          if(result.deletedCount===0){
53              return res.status(404).json({message:"No se encontro
                    ese estudiante"});
54          }
55          res.status(204).json({message:"Estudiante eliminado",
               result})
56      } catch (error) {
57           res.status(500).json({ message: "Error al eliminar",
               error: error.message });
58
59      }
60  }
```

2.2.6. Sección principal

Finalmente el index.js, donde está la entrada principal del proyecto, en el se tiene el uso de express. Se instancia la clase y la llamamos app. Dentro de ella, debe de contener el uso explícito de json. El mensaje de entrada es un hola. Nuestros endpoints son los siguientes

- /students
- /insert
- /update
- /delete

Al llamar a estos endpoints llama a las funciones que se describieron en el controlador. Al ser dispuestas estas funciones, tenemos la función run que nos permite el conectarnos a nuestra base de datos y al final, liberamos nuestro link. En este caso, localhost y con el puerto 3000. Tómese en cuenta que cuando se crea el contenedor, se generará una IPv4 para nuestro servicio, y por lo tanto si tendrá su localhost en base a ese contenedor y a lo que le asigne nuestro switch virtual

Capítulo 3

Resultados

Por lo tanto se probó este proyecto mediante curl, con 4 sencillos comandos.

3.1. El hola

El primer comando que se prueba es la conexión directa, por lo tanto solo esperamos un hola como mensaje en la figura 3.1

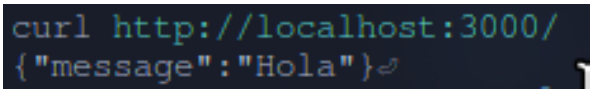


Figura 3.1: Respuesta Inicial

3.1.1.



(a) Creación



(b) Lectura



(c) Actualización



(d) Eliminación

Figura 3.2: Grid 2×2 de fotos

En la figura 3.2, tenemos el cuarteto de las siglas del CRUD. La figura 3.2a, posee la creación de un nuevo estudiante, El primer intento fue con un estudiante que ya existía,

devolviendo el error de que no se puede volver a ingresar un estudiante. Subsecuente a eso, está el intento correcto, en el que el estudiante es diferente y se agrega exitosamente. Para la figura 3.2b, se encuentra la parte de lectura, que devuelve todos el conjunto de estudiantes ingresados. En la figura 3.2c se actualiza un estudiante. EL primer caso se coloca un correo diferente y da el error de que el estudiante con tal correo no existe, mientras que en el segundo, se coloca el correo viejo y el correo nuevo y da el resultado de que se actualizó con éxito el correo. Finalmente en la Figura 3.2d, se encuentra la eliminación. Se coloca un correo erróneo y da el error de que no se encontró ese estudiante. Mientras que ahora, se coloca el nombre de ese estudiante y da el resultado exitoso de la eliminación.

Esto mismo se puede realizar con postman u otras herramientas para el testeo de API's

Capítulo 4

Conclusiones

La gran mayoría de proyectos modernos se utilizan contenedores, resultando en una facilidad en el despliegue y el evitar la famosa frase de *En mi computadora sí funcionaba*. En este trabajo se vieron conceptos como modelo controlador, api rest, programación asíncrona, ORM, y sobre todo, bases de datos no relacionales